A Solver-Computed Chunk Partition for Atomic's CDN Build

Modelling, solving and measuring how a lazily loaded component library splits its JavaScript

Yassine Lakhdar
Coveo

November 3, 2026

Abstract

Coveo Atomic is a library of 206 lazily loaded web components served from a CDN. Its rspack build split code by module, so a typical search page fetched 162 JavaScript files; ADR 0001 then grouped widely shared modules into one chunk with a hand-picked threshold, `minChunks: 10`, which halved that count. We ask whether a computed partition does better. Modules are grouped by the set of pages that use them, and an integer program decides which groups share a chunk, minimising a per-page cost of bytes and requests whose weights are fitted on the ADR's own 28 mobile measurements ($R^2 = 0.9996$; one request costs as much as 0.91 KB). On 42 page shapes drawn from the repository, the solved partition cuts the mean page from 83.1 to 14.0 files and from 187.2 to 160.5 KB gzipped; trained on 12 pages and tested on 30 unseen ones, it still removes 78 % of the files with bytes flat. Greedy and agglomerative heuristics reach the solver's objective within 0.2 %, size balancing in the style of LPT scheduling is 53 to 95 % worse, and a one-file-per-page bound leaves at most 0.09 s. Measured with the ADR's harness under Lighthouse's mobile throttling, the partition makes the ADR's four pages ready 39, 300, 605 and 453 ms sooner than `minChunks: 10` (1 to 13 %), within 140 ms of the model. Measurement also exposed two failure modes that a model of files and bytes cannot see: grouping external bundles delays them, which made one page 274 ms slower, and a partition goes stale within days as components change.

1 Introduction

Coveo Atomic is a library of 206 web components that customers load from a content delivery network (CDN). A page includes one entry script. When the page uses a component, the entry's runtime looks the tag up in a table, fetches the chunks that the component's code needs, and defines the element. Each Atomic release is deployed under its own CDN path, so no file is reused across releases and every first visit pays for every request.

Until October 2026 the CDN build used rsbuild's `split-by-module` strategy with no minimum chunk size: every npm package became a chunk, and every set of modules shared by the same components became a chunk of its own. Atomic's components share many small modules, such as Lit, base classes, controllers and utilities, so a typical search page fetched 162 Atomic files, 109 of them under 2 KB [1]. ADR 0001 [1] measured six alternatives on four page shapes with a dedicated harness under Lighthouse's mobile throttling, and chose to group the modules that at least ten chunks use into one shared chunk. The typical page dropped to 76 files, and its time until ready from 4.21 s to 3.66 s.

A threshold is a one-parameter family of partitions. The underlying decision, which modules travel together in which file, is combinatorial: it trades the bytes a page downloads but never

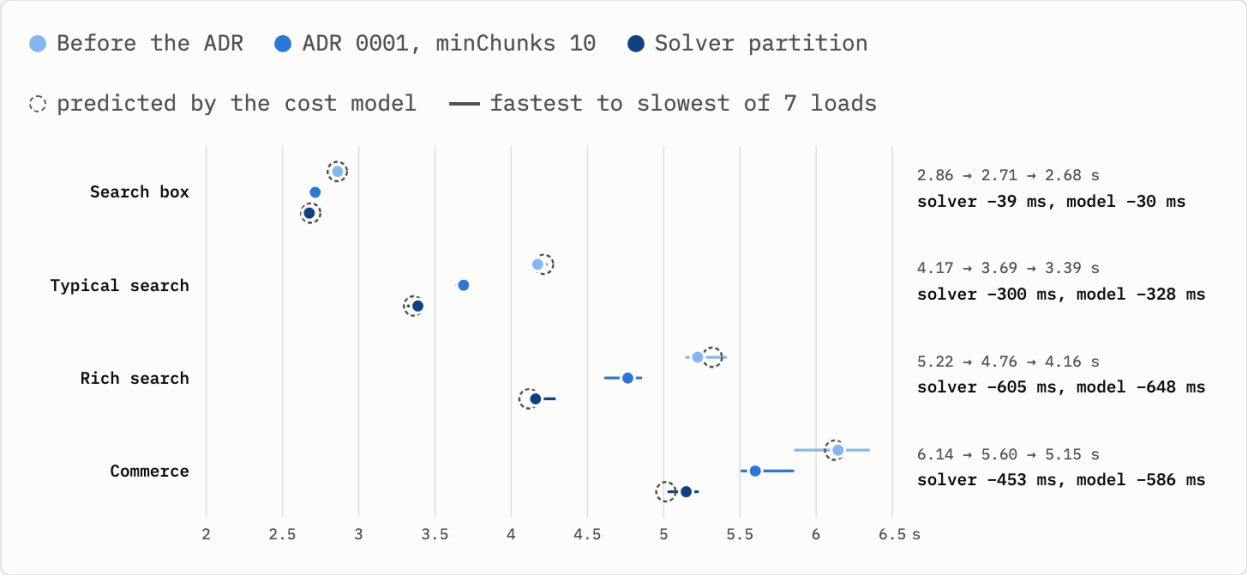


Figure 1: Measured time until ready on Lighthouse's mobile profile for the ADR's four pages and three builds of one commit (run 5 of Section 6.7): median of seven interleaved loads, with the fastest and slowest load drawn as a line. Dashed rings show what the cost model predicted from the `minChunks: 10` measurement.

runs against the number of requests it makes. This paper reports a prototype that computes that decision instead of choosing it, the experiments that tested it, and what measuring it taught. Figure 1 shows the outcome on the ADR's own pages. Our contributions are:

1. a formulation of chunk assignment as a partition of usage-signature groups that minimises bytes plus a fitted price per request, with a proof that grouping modules by signature loses nothing (Section 3);
2. a cost model fitted on the ADR's measurements (Section 4) and a pipeline that enforces any partition in rspack without touching the entry files (Section 5);
3. experiments on 42 page shapes: in-sample and held-out results, a sensitivity analysis, a comparison with greedy, agglomerative and LPT-style heuristics and with a lower bound, synthetic training sets, and five runs of the ADR's harness (Section 6);
4. two failure modes that only measurement revealed, and a way to ship the partition as data (Sections 7 and 8).

2 Background

2.1 Atomic's CDN build

The CDN build is an rsbuild configuration on top of rspack. It has three entry files: `atomic.esm.js`, the loader most pages include, `index.esm.js` and `index.js`. Components load through dynamic imports compiled into a table that maps each tag to an asynchronous function; the function awaits one request per chunk that the component's code needs. Headless, the library that holds Atomic's state and calls the Coveo APIs, ships as four bundles (search, commerce, insight and recommendation) and the validator Bueno as one; the build treats these five as externals imported by their

Table 1: ADR 0001's mobile measurements: time until ready, with the number of Atomic files in parentheses [1]. A is split-by-module alone, B rspack's defaults, C a shared chunk of the modules used by at least N chunks, E option C extended to npm packages, and F one shared chunk per line of business. E and F come from the ADR's second, seven-load measurement, in which C with $N = 10$ took 2.66, 3.65, 4.62 and 5.52 s.

Option	Search box	Typical search	Rich search	Commerce
A	2.91 s (65)	4.21 s (162)	5.27 s (225)	6.09 s (196)
B	2.53 s (15)	4.05 s (50)	5.59 s (92)	5.89 s (67)
C, $N = 5$	2.77 s (23)	3.60 s (50)	4.55 s (97)	5.42 s (78)
C, $N = 10$ (chosen)	2.68 s (30)	3.66 s (76)	4.66 s (134)	5.54 s (108)
C, $N = 20$	2.66 s (34)	3.85 s (106)	4.86 s (168)	5.76 s (142)
E	2.64 s (28)	3.63 s (74)	4.63 s (132)	5.50 s (105)
F	2.62 s (28)	3.63 s (75)	4.63 s (134)	5.50 s (107)

versioned CDN path. Two cache groups shape the asynchronous chunks on `main`: `atomic-shared`, holding the modules of Atomic's sources and of the Lit runtime that at least ten chunks use, and one chunk per dayjs locale.

2.2 ADR 0001 and its harness

Table 1 summarises the ADR's mobile measurements. The harness serves four pages (a search box alone, a typical search page, a rich search page and a commerce page) over HTTP/2 with gzip from a local server that lays files out like the CDN and replays recorded API answers, so only the Atomic files differ between builds. Every load starts with an empty cache in a fresh browser context. Throttling goes through the Chrome DevTools Protocol with Lighthouse's "Slow 4G" mobile profile [2]: 150 ms of latency, 1.6 Mbps down, 750 kbps up and a CPU four times slower. A desktop profile uses 100 ms and 5 Mbps. A page is *ready* when every Atomic tag on it is defined and its results or products have rendered, or, for the search-box page, when the box's input exists. The ADR reports medians of five mobile or three desktop loads, interleaved across builds.

2.3 Why requests cost time

HTTP/2 multiplexes requests over one connection [9]. That removes head-of-line blocking between requests, not the work each one implies: headers, scheduling, and a chunk's own wrapper and compression framing. Small files also compress worse than large ones. Khan Academy found that serving individual files over HTTP/2 instead of packages made its pages slower, mainly through lost compression [6]. Next.js adopted "granular chunking", which bounds shared chunks to at least 20 KB and at most 25 requests per page [7]. Lighthouse names the length of critical request chains and their bytes as the two levers [3], and parse and compile time grows with the bytes of script, two to five times more slowly on average phones than on fast ones [5]. For scale, the median mobile page in 2024 made 22 JavaScript requests for 558 KB [4]; a typical Atomic page alone makes 74.

3 Problem formulation

3.1 Pages, modules and usage

Let $\mathcal{P}$ be a set of pages, each described by the Atomic tags it uses. For a tag, the closure of static imports from its component module in the build's module graph gives the modules the tag needs,

and a page needs the union over its tags. Modules that the entry chunks of `atomic.esm.js` contain are always loaded and leave the problem. A module is *movable* if the build already places it in an asynchronous chunk; the five externals are kept where the build puts them (Section 6.7 explains why). For a movable module m, let $U(m) \subseteq \mathcal{P}$ be the pages that need it, and r_m its raw size.

3.2 Usage-signature groups

Modules with the same usage set form a *group*, $G_S = \{m : U(m) = S\}$, with raw size $r_g = \sum_{m \in g} r_m$. On the 42 pages of Section 6, 662 movable modules fall into 127 groups; 192 movable modules are reached from no page and keep the build's existing rules.

Lemma 1. *Under the cost of Section 3.3, some optimal partition keeps every group in one chunk.*

Proof. Let m_1 and m_2 have $U(m_1) = U(m_2) = S$ and sit in different chunks A and B, and let $L(k)$ be the set of pages that load chunk k. Every page of S loads both chunks. Moving m_2 into B changes the cost by $\beta r_{m_2}\big(|L(B) \setminus S| - |L(A) \setminus S|\big)$, and pages of S that no longer need A save a request. Moving m_1 into A changes it by $\beta r_{m_1}\big(|L(A) \setminus S| - |L(B) \setminus S|\big)$, with the same kind of saving. Pages outside S load A and B for other modules, so neither move changes which chunks they load. One of the two moves therefore never increases the cost, and repeating it gathers each group in one chunk. □

3.3 Cost of a partition

A partition assigns each group g to a chunk $k(g)$. Chunk k has raw size R_k and, by a linear fit over the build's asynchronous chunks, gzipped size $\alpha + \beta R_k$ with $\alpha = 422$ bytes and $\beta = 0.139$. Page p loads $L(p) = \{k(g) : p \in U(g)\}$, and its cost in gzipped-byte equivalents is

$$c(p) = \sum_{k \in L(p)} \big(\alpha + \lambda + \beta R_k\big),$$

where λ is the price of one request expressed in bytes (Section 4). The objective is $\sum_p w_p\, c(p)$, with $w_p = 1$ unless stated. The bytes a page truly needs, $\beta \sum_{g : p \in U(g)} r_g$, do not depend on the partition, so minimising c trades the bytes a page downloads but does not need, its *waste*, against its number of files. In seconds, the same cost reads $t(p) = b \cdot \mathrm{KB}(p) + c \cdot \mathrm{files}(p)$ with the coefficients of Section 4.

3.4 Integer program

Groups are indexed $0, \dots, n-1$ by decreasing size, and chunk k is opened by group k. A binary $x_{g,k}$, defined for $k \le g$, puts group g in chunk k; a binary $y_{p,k}$ says that page p loads chunk k; an integer $w_{p,k} \ge 0$ is the waste page p downloads in chunk k. With $z_g = \beta r_g$ and $F = \alpha + \lambda$:

$$\begin{aligned}
\min \quad & \textstyle\sum_p \sum_k \big(w_{p,k} + F\, y_{p,k}\big) && \\
\text{s.t.} \quad & \textstyle\sum_{k \le g} x_{g,k} = 1 && \forall g, \\
& x_{g,k} \le x_{k,k} && \forall k < g, \\
& x_{g,k} \le y_{p,k} && \forall p,\ \forall g \text{ with } p \in U(g),\ \forall k \le g, \\
& w_{p,k} \ge \textstyle\sum_{g \ge k,\ p \notin U(g)} z_g\, x_{g,k} - W_{p,k}\,(1 - y_{p,k}) && \forall p, k,
\end{aligned}$$

where $W_{p,k}$ is the sum of the z_g in that constraint, a big-M. The needed bytes are a constant added back when reporting. Restricting $x_{g,k}$ to $k \le g$ and requiring an opened chunk breaks the

symmetry between chunk labels. For 127 groups and 42 pages the model has 8,128 x, 5,334 y and 5,334 w variables. We solve it with CP-SAT [10, 11] from OR-Tools 9.15, with eight workers, a 150 s limit, and the hint that every group is alone.

3.5 A lower bound

No partition does better for a page than one file holding exactly the bytes it needs, so $c(p) \geq \alpha + \lambda + \beta \sum_{g:p \in U(g)} r_g$, even if modules are duplicated across files. Averaged over the 42 pages this floor is 0.736 s, against 0.83 s for the solved partition at model level: no strategy within this model, duplication included, can save more than 0.09 s per page.

4 Cost model

The ADR measured 28 combinations of build and page for which both files and bytes are known: options A, B, C with $N = 5$, 10 and 20, E and F, on four pages. A least-squares fit of

$$t = a_{\text{page}} + b \cdot \text{KB} + c \cdot \text{files},$$

with one intercept per page, gives $b = 5.0$ ms per gzipped KB and $c = 4.6$ ms per file, with $R^2 = 0.9996$ and a largest residual of 65 ms. The byte coefficient matches the link: 1 KB takes 5.1 ms at 1.6 Mbps. The ratio $\lambda = c/b = 0.91$ KB is the price of a request in bytes. Without option B, whose duplicated code makes it an outlier in bytes, the fit gives $b = 5.8$ ms, $c = 4.4$ ms and $\lambda = 0.76$ KB. The gzip model $\alpha + \beta R$ is a separate linear fit over the 427 asynchronous chunks of the `minChunks: 10` build, so every file also costs about 0.4 KB of wrapper.

The model has no CPU term separate from bytes, no term for the depth of request chains, and per-page intercepts that come from the ADR's pages; the desktop profile is not modelled. Section 6.7 shows that the missing chain term matters.

5 Prototype pipeline

Figure 2 shows the pipeline. All of it lives in one folder of a prototype branch (Appendix A).

1. **Pages.** `pages.mjs` extracts the Atomic tags of 42 page shapes from the repository: 7 Storybook sample pages, 4 CDN smoke-test pages, 30 pages of the sample applications (React, Vue, Angular, Next.js, Vite, and a hybrid of Headless and Atomic), and the ADR's search-box-only shape.

2. **Analysis build.** A build with module concatenation turned off makes every module visible in the stats. `analyze.mjs` computes the closures, the movable modules, the groups and the gzip fit, and writes the model.

3. **Fit and solve.** `fit.py` fits the cost model on the ADR's measurements; `solve.py` builds and solves the integer program and writes a JSON map from module path to chunk name.

4. **Enforced build.** `rsbuild.proto.mjs` turns the map into one cache group per chunk, each with `enforce: true`, `chunks: 'async'`, priority 50, and a test that looks the module up in the map. Modules missing from the map follow the existing groups, and the entry files are unchanged.

5. **Measurement.** `measure.mjs` parses the compiled lazy-loading table, maps chunk ids to files through the stats, gzips each file at level 9, and reports files, bytes and modelled time per

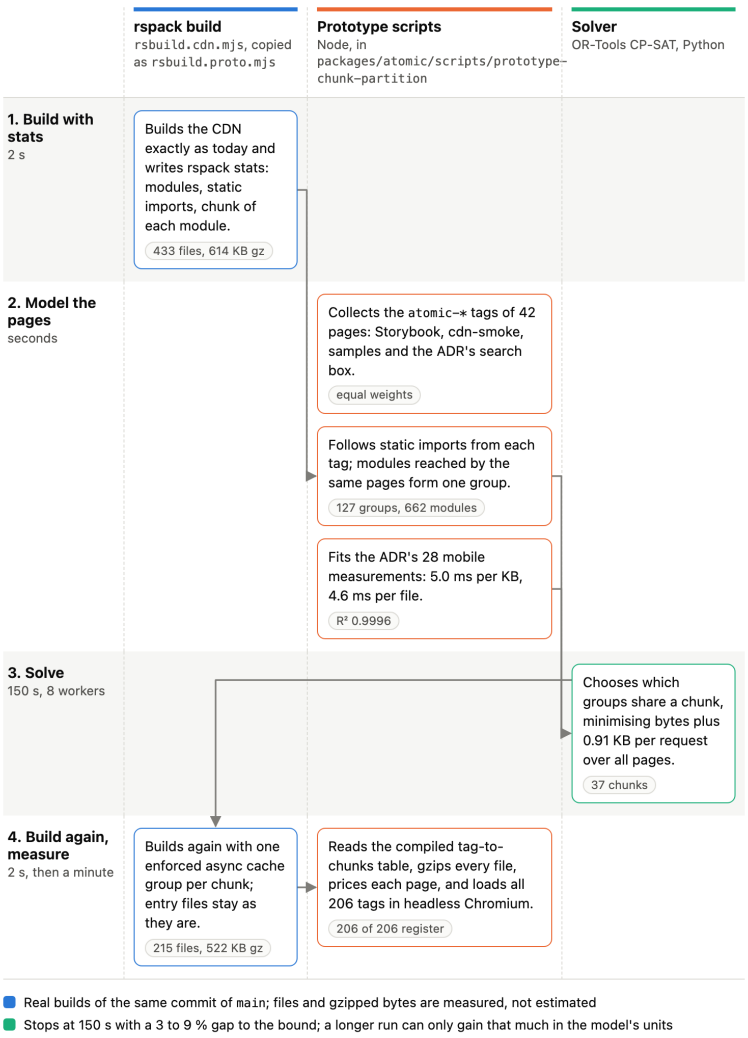


Figure 2: The prototype pipeline: one real build with stats, a model of 42 page shapes, a solve, then a second real build that enforces the partition and is measured.

page. `check-registration.mjs` loads all 206 tags in headless Chromium against the build and checks that every one registers.

Iterations before the final solves. The first enforced build fetched 28.4 files per page where the model predicted 9.5. Two causes explained the gap. Rspack's module concatenation merged modules into concatenated modules whose identifiers the map did not contain, fixed by computing the model on the unconcatenated analysis build and mapping by inner module path. Modules with a copy in the initial chunks of `index.esm.js` were wrongly treated as always loaded, fixed by treating only the entry chunks of `atomic.esm.js` that way. After these fixes the enforced build fetched 14.0 files per page. The remaining difference with the model's 9.6 comes from the entry files and the modules the map does not place. Figure 3 shows what a typical page downloads in each build.

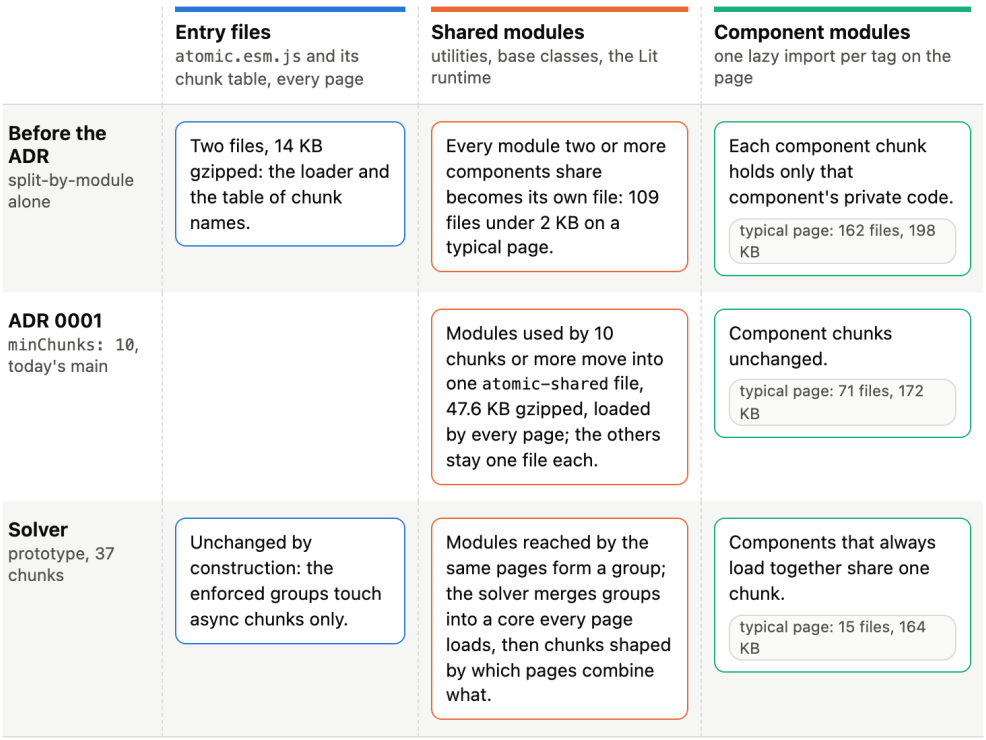


Figure 3: What a page downloads in each build, for the typical sample page `samples/atomic/search-vite`.

6 Experiments

6.1 Setup

All experiments ran on one Apple Silicon Mac with 14 logical cores under macOS 26, with Node.js 24.21, Python 3.9.6, OR-Tools 9.15.6755, and Chromium from Playwright. Unless stated otherwise, builds come from `main` of October 31, 2026. Files and gzipped bytes of real builds are exact; times are modelled as in Section 4, except in Section 6.7, where they are measured. Sections 6.5 and 6.6 evaluate partitions at model level, which counts neither the entry files nor the modules outside the map.

6.2 In-sample results

Table 2 compares the three builds over the 42 page shapes. CP-SAT stops at its time limit with a feasible partition of 37 chunks, 7.8 % above its lower bound. The mean page fetches 14.0 files instead of 83.1, and 160.5 KB instead of 187.2. Of the whole improvement from the pre-ADR build to the solver build, the ADR captured 55 % of the files and 44 % of the bytes (Figure 4). Figures 5 and 6 show every page; the solver build downloads fewer bytes than the ADR build on every page but two. In modelled time (Figures 7 and 8), the ADR removed half of the request time and the solver nearly all of it. Rebuilt on November 3 with the externals kept apart (Section 6.7) and a fresh map, the solver build fetches 14.7 files and 163.6 KB per page, against 83.1 and 188.6 for the ADR build of the same commit.

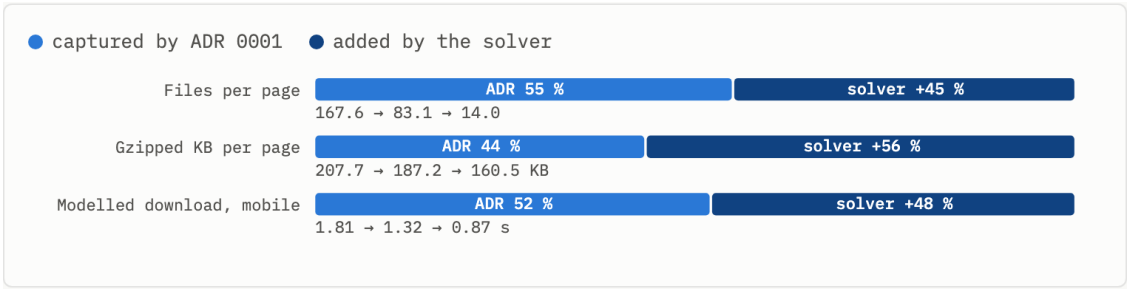


Figure 4: The whole improvement from the pre-ADR build to the solver build, split between what the ADR captured and what the solver adds.

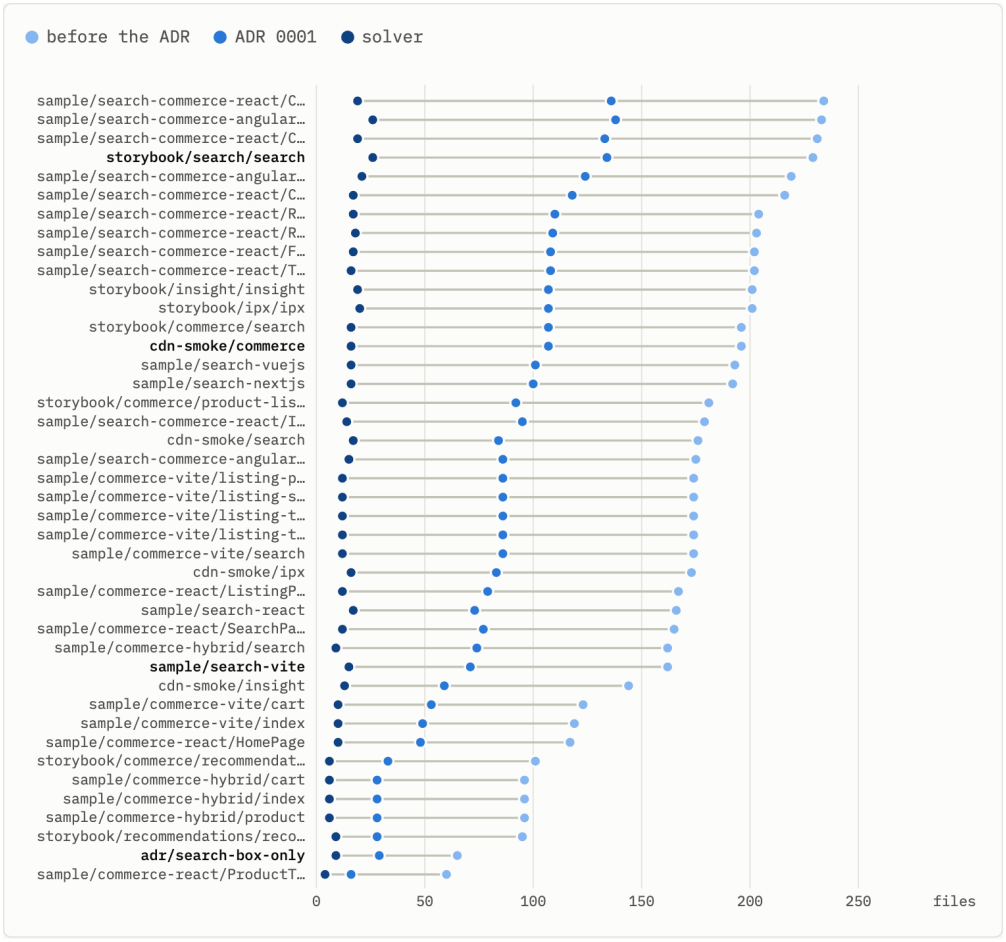


Figure 5: Files a first visit fetches, entry included, for each of the 42 page shapes and the three builds, sorted by the pre-ADR build. Bold rows stand in for the ADR's four shapes.

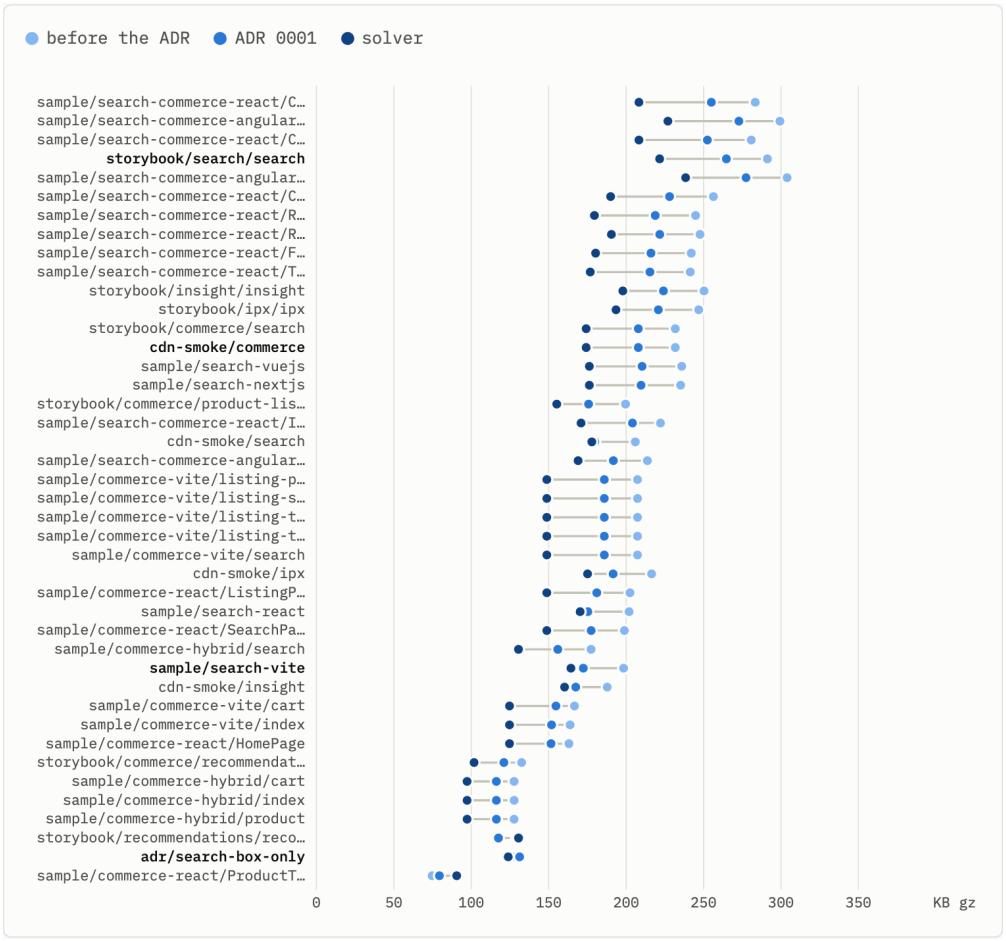


Figure 6: Gzipped bytes per page. The ADR's dot sits right of the pre-ADR one on small pages, where it traded bytes for requests; the solver's dot is left of both on every page but two.

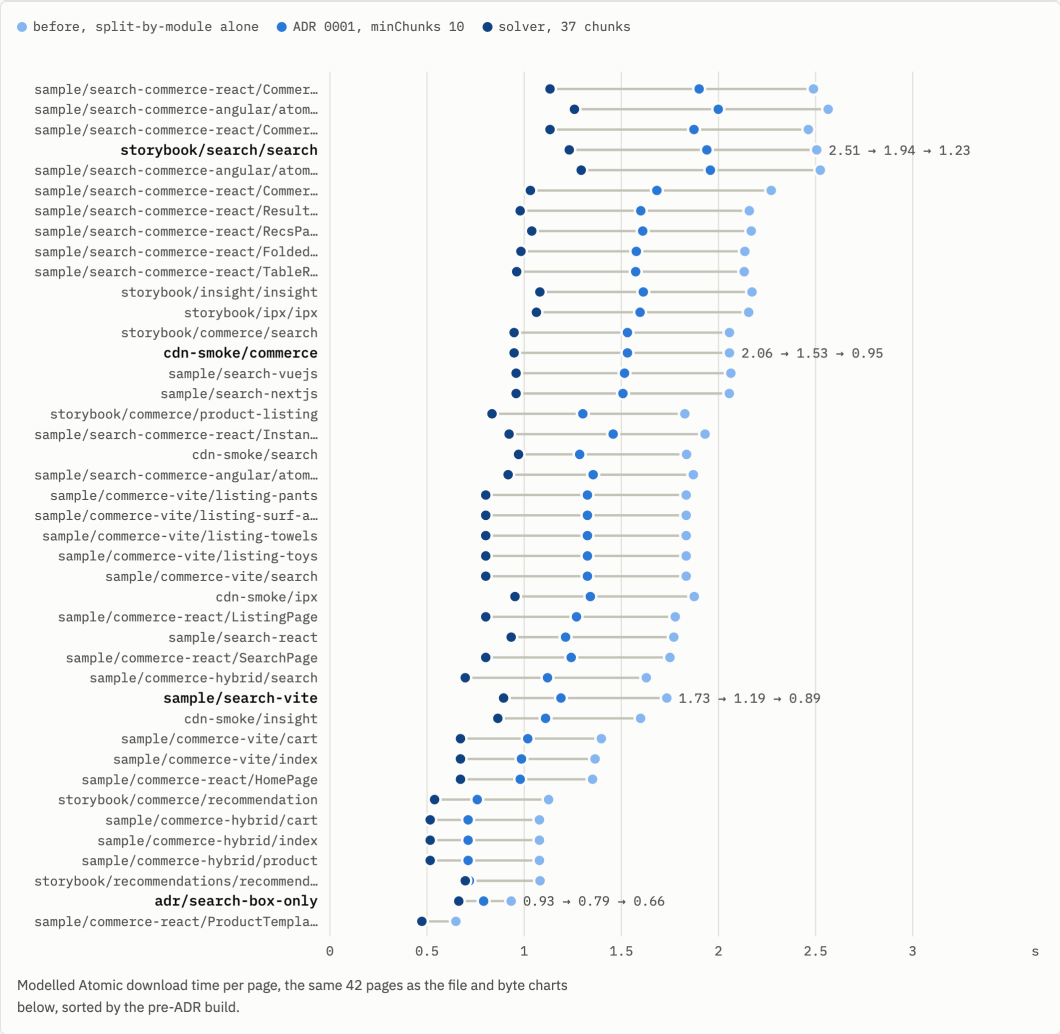


Figure 7: Modelled Atomic download time on the mobile profile for each of the 42 page shapes and the three builds of October 31, sorted by the pre-ADR build.

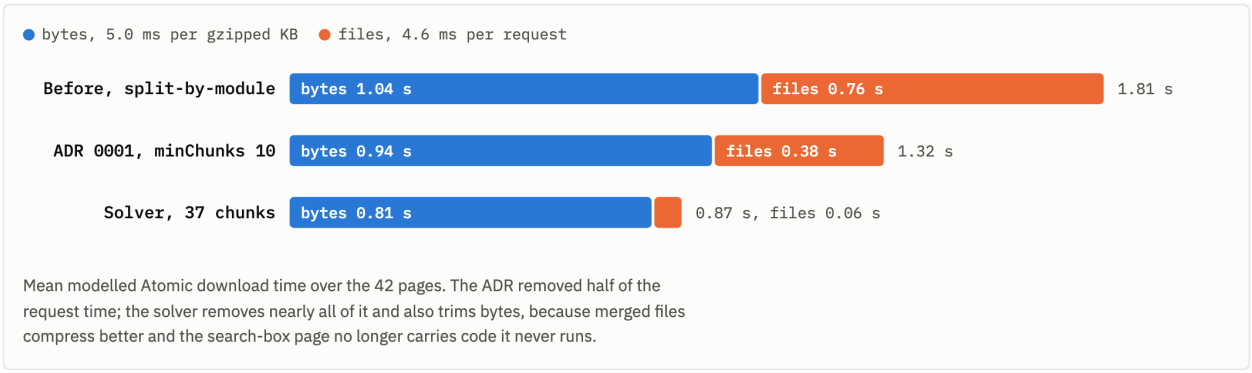


Figure 8: The Atomic part of the modelled time, mean over the 42 pages, split between bytes and requests. The ADR removed half of the request time; the solver removes nearly all of it and trims bytes too.

Table 2: Means over the 42 page shapes, builds of October 31. The last column counts the JavaScript files of the whole CDN folder.

Build	Chunks	Files	KB gzipped	Modelled time	Folder files
Before the ADR (split-by-module)	–	167.6	207.7	1.81 s	532
ADR 0001, `minChunks: 10`	–	83.1	187.2	1.32 s	433
Solver partition	37	**14.0**	**160.5**	**0.87 s**	215

Table 3: Held-out test: builds measured on the 30 sample pages. The second solver was trained on these pages, the first was not.

Build	Chunks	Files	KB gzipped	Modelled time
Before the ADR	–	169.4	209.1	1.82 s
ADR 0001	–	84.0	188.4	1.33 s
Solver trained on the 12 other pages	24	**18.9**	192.1	**1.05 s**
Solver trained on all 42 pages	37	13.6	158.5	0.86 s

6.3 Held-out test

A partition could memorise its training pages. We therefore solved again on the 12 Storybook, smoke-test and ADR pages only, built the result, and measured it on the 30 sample pages it had never seen (Table 3). Out of sample the gain comes from requests: files drop by 78 %, bytes rise by 2 %, and 17 of the 30 pages download more bytes than with the ADR build, the worst by 34.5 KB. The modelled time still falls from 1.33 to 1.05 s, against 0.86 s when the same pages are part of training.

6.4 Sensitivity to the price of a request

Halving or doubling λ changes how many chunks the solver opens, not the outcome (Table 4): every setting beats the ADR build on files and bytes at once, and the modelled time moves by 10 ms.

6.5 Algorithms

Table 5 compares strategies on the same objective. Longest-processing-time-first (LPT) scheduling assigns jobs in decreasing size to the least-loaded of m machines, a $4/3 - 1/(3m)$ approximation for the makespan [12]. Its goal is balance, and balance does not matter here: what matters is which pages share a chunk. Balancing the groups over 37 or 14 chunks costs 53 and 95 % more than CP-SAT. The greedy heuristics place groups one at a time into the existing or new chunk with the least marginal increase of the objective; agglomerative merging starts from one chunk per group and repeatedly merges the pair that lowers the objective most. Three of them land within 0.2 % of CP-SAT, all of them together in about a second, and a 900 s CP-SAT run warm-started from the 150 s solution improves it by only 0.3 %. The solver's 7.8 % gap therefore measures a weak bound rather than room left in the solution. On the 12-page training split of Section 6.3 the greedy and agglomerative heuristics land within 0.1 to 1.0 % of CP-SAT and generalise as well: 1.005 s on the 30 held-out pages, against 1.013 s for CP-SAT.

Table 4: Sensitivity to λ, means over the 42 pages, real builds.

λ (KB per request)	Chunks	Files	KB gzipped	Modelled time	Gap after 150 s
0.45 (half the fit)	42	17.1	160.2	0.88 s	7.0 %
0.91 (the fit)	37	14.0	160.5	0.87 s	7.8 %
1.81 (double the fit)	25	13.3	163.6	0.88 s	9.3 %

Table 5: Strategies on the solver's objective, 42 pages, model level. The floor gives each page one file holding exactly what it needs.

Strategy	Chunks	vs CP-SAT	Files	KB gzipped	Modelled time
Signature groups, no merging	127	+27.7 %	50.3	165.8	1.062 s
LPT size balancing, 37 chunks	37	+53.5 %	25.1	231.2	1.276 s
LPT size balancing, 14 chunks	14	+95.0 %	12.4	311.4	1.621 s
Greedy, largest group first	32	−0.2 %	7.9	158.0	0.830 s
Greedy, smallest group first	34	+3.3 %	8.8	162.9	0.859 s
Greedy, most-used group first	31	−0.2 %	7.7	158.2	0.830 s
Agglomerative merging	36	−0.2 %	9.4	156.6	0.830 s
CP-SAT, 150 s	37	0	9.6	156.8	0.832 s
CP-SAT, 900 s, warm start	33	−0.3 %	8.2	157.5	0.829 s
CP-SAT lower bound		−7.8 %			
Floor: one file per page			1.0	145.5	0.736 s

6.6 Synthetic training sets

The held-out gap of Section 6.3 raises the question of whether training on more varied pages helps. We added to the 12 training pages either 48 random sub-pages of them (each tag kept with probability 0.4 or 0.7, interface tags always kept, weight 0.5) or one page per component (206 pages, weight 0.1), solved for 300 s with at most 48 chunks, and scored each partition on the 30 held-out pages (Table 6). Random sub-pages recover 22 ms; one page per component makes the held-out pages 108 ms slower, because the solver then optimises for pages that no real page resembles. Real page compositions are the remaining lever on generalisation.

6.7 Harness measurements

All the times above are modelled. We then ran the ADR's harness on its four pages, copied from the ADR's scripts with three adjustments: a new local TLS certificate, Playwright resolved from the main checkout, and the themes, language files and icons that the prototype builds lack, copied from the CDN build. Each run interleaves its builds in shuffled order on each page, with seven mobile and three desktop loads per build and page, after one unthrottled load per build and page that checks errors and file counts. Table 7 lists the five runs; Figure 9 compares measured and modelled gains.

Run 1: externals grouped. The first measurement contradicted the model: the solver build was 274 ms slower on the typical search page and no faster on the rich page, despite fetching 16 files instead of 74 and fewer bytes. A request waterfall (Figure 10) showed the cause. Treating each external as a 42-byte module, the analysis had merged the five externals into solver chunks. On `main`, each external sits alone in a 160-byte chunk, so the Headless download starts early, in

Table 6: Training sets, scored at model level on the 30 held-out pages and on the 12 training pages.

Training set	Files	KB gzipped	Modelled time	Training pages
ADR 0001 build (no training)	80.3	187.3	1.307 s	1.263 s
12 pages	13.1	189.8	1.013 s	0.789 s
12 pages and 48 random sub-pages	16.7	182.1	0.991 s	0.823 s
12 pages and 206 one-component pages	17.7	207.2	1.121 s	0.915 s
All 42 pages (in sample)	7.7	156.5	0.821 s	0.848 s

Table 7: The five harness runs: mobile gain of the solver build over the `minChunks: 10` build, median of seven loads, in ms for the search box, typical, rich and commerce pages.

Run	Builds and source	Measured gain	Modelled gain
1	three builds of October 31, externals grouped	+43, +274, +7, −524	−128, −321, −629, −583
2	`main` of November 2, map of October 31	−5, −317, −489, −500	−24, −297, −600, −554
3	builds from two commits (not used)	−43, −317, −555, −459	–
4	one commit of November 3, fresh map	−28, −307, −510, −502	−30, −328, −648, −586
5	same commit, three builds	−39, −300, −605, −453	−30, −328, −648, −586

parallel with everything else. In the solver build, the import of the Headless search bundle sat in a 73 KB chunk and started only once that chunk had arrived, about 0.75 s later; a 2.7 KB chunk that the page loads carried the import of the 139 KB recommendation bundle, which this page never uses, and that download came first. The commerce page escaped both effects and gained 524 ms. The fix keeps externals out of the groups, so they stay where the build puts them.

Run 2: a two-day-old map. With the externals apart and the map of October 31 applied to the source of November 2, the three content pages gained 317 to 500 ms, within 111 ms of the model, but the search box gained 5 ms (permutation $p = 0.13$). Between the two dates the search box's code had gained a dependency, a props-validation module of 3.6 KB before compression, that the old map placed in a 15 KB chunk; the search-box page downloaded that chunk for one module.

Runs 3 to 5. Between runs 2 and 3 the build checkout changed branches, so later builds come from `main` of November 3 plus five commits to the custom-elements manifest build. Run 3 compared a solver build from that commit with an ADR build from the previous one and is not used. Runs 4 and 5 compare builds of one commit with a map computed on it; run 5 adds the pre-ADR build (Table 8, Figures 1 and 11). The two runs agree within 95 ms on every page.

Validation of the harness and the model. The harness reproduces the ADR: in run 5 the pre-ADR build lands within 54 ms of the times the ADR published for option A, and the ADR build within 105 ms of option C with $N = 10$, nearly three weeks and many commits later. The model predicted the pre-ADR build's loss within 25 ms in run 1 and within 96 ms in run 5, and the solver's gain within 140 ms in runs 4 and 5. On the three content pages the measured gains are 77 to 94 % of the modelled ones: the model slightly overstates the largest gains. An exact permutation test on the difference of medians reaches its smallest attainable value, $p = 0.012$, for every page in runs 4 and 5, including the 39 ms of the search box.

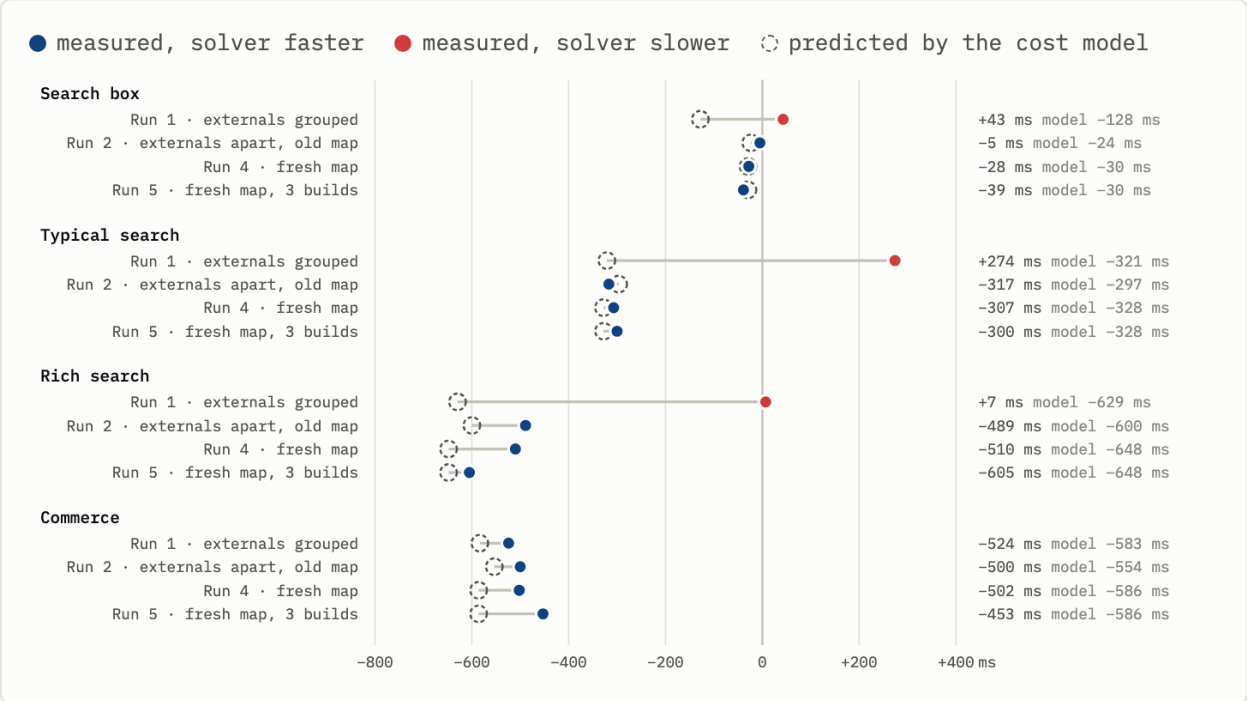


Figure 9: Measured gain of the solver build over `minChunks: 10` in each usable run, against the model's prediction for the same builds. Run 1 grouped the externals, run 2 used a map computed two days before its source.

Table 8: Run 5, mobile: median time until ready of seven loads, with the fastest and slowest load, and the exact permutation p of the solver's difference with `minChunks: 10`. With seven loads per build, 0.012 is the smallest attainable value. Files are the Atomic files fetched by the pre-ADR, ADR and solver builds.

Page	Before the ADR	`minChunks: 10`	Solver	Gain	Model	p	Files
Search box	2862 (2827–2919)	2715 (2680–2719)	2676 (2641–2688)	−39	−30	0.012	67, 30, 12
Typical search	4174 (4143–4235)	3688 (3641–3728)	3388 (3325–3432)	−300	−328	0.012	163, 74, 16
Rich search	5223 (5150–5408)	4765 (4618–4851)	4160 (4125–4284)	−605	−648	0.012	226, 132, 29
Commerce	6144 (5863–6344)	5601 (5513–5848)	5148 (5033–5223)	−453	−586	0.012	197, 108, 17

7 Discussion

Where the gain comes from. Requests dominate. On the typical page the solver cuts 73 files to 15 and 176 KB to 164 KB, as counted by the measurement script; at 4.6 ms per file and 5.0 ms per KB, the files are worth 264 ms and the bytes 64 ms of the modelled 328 ms, against 300 ms measured. Bytes fall as well, because the single shared chunk of the ADR carried code that many pages never run.

What a model of files and bytes cannot see. The externals problem was invisible to the model. Grouping an external moved its download behind a whole chunk: the cost of a request depends on where it sits in the chain, not only on how many requests there are. The waterfalls also show a lever outside the partition: in every build, the second wave of component files starts only once the 179 KB Headless search bundle has arrived, so Headless sits on the critical path of every page. Preloading it is a separate optimisation that we did not measure.

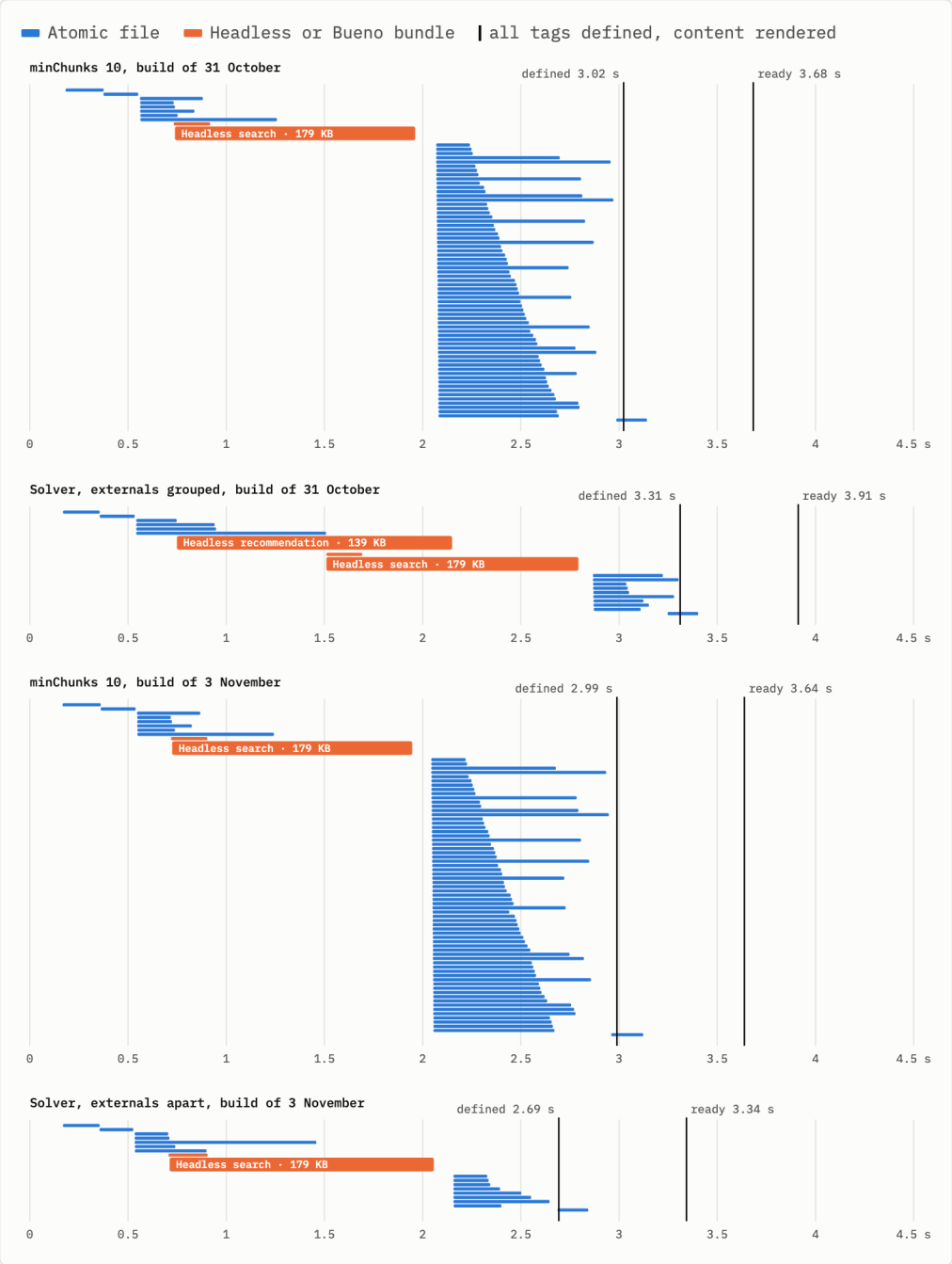


Figure 10: Requests of the typical search page, one mobile-throttled load per build, with the time all tags were defined and the time results rendered. With the externals grouped (second panel), Headless waits for a 73 KB chunk and the unused recommendation bundle loads first. The recorder captured scripts only, not API calls.

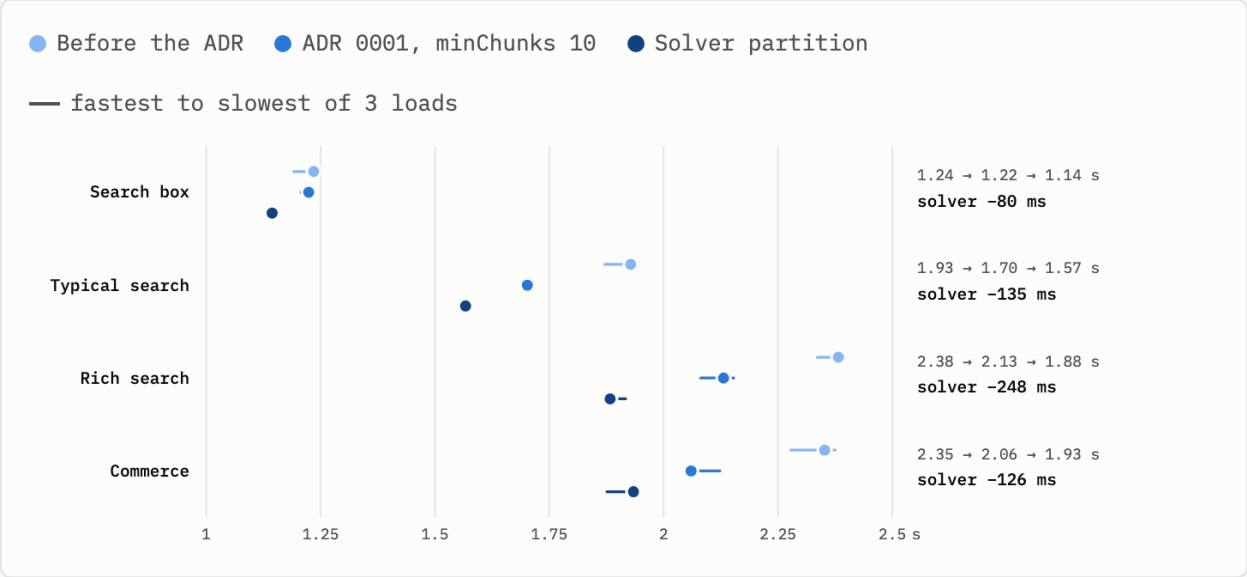


Figure 11: Run 5 on the desktop profile (100 ms, 5 Mbps): median of three loads, with the fastest and slowest load. The solver saves 80, 135, 248 and 126 ms.

Generalisation. Trained on repository pages and tested on others, the partition keeps most of its request savings but not its byte savings (Section 6.3), and synthetic pages do not close that gap (Section 6.6). The population that matters is customers' pages, weighted by traffic; their compositions are the next input to collect.

Staleness. The map is data computed from the module graph at one commit. Two days of ordinary work changed the search box's closure enough to cancel its gain (run 2). Any deployment must regenerate the map when components change, or detect that it is stale.

Algorithm choice. The algorithm is not the lever. A one-second greedy heuristic matches CP-SAT within 0.2 %, and even a bespoke file per page leaves at most 0.09 s on the table. CP-SAT remains useful as a check: its bound and a long run show how far a heuristic solution is from the best one.

Threats to validity. Page shapes are repository proxies with equal weights. All measurements come from one machine with emulated throttling, a local server and recorded API answers, like the ADR's, and only first visits are measured; within a release, repeat visits hit the HTTP cache. Each comparison rests on seven loads per build and page, and the same builds differed by up to 95 ms between runs 4 and 5. Builds come from two commits across the runs, which Table 7 keeps apart. Every build registers all 206 components, but the CDN smoke tests and the ADR's other invariants did not run on the solver builds.

8 Deployment proposal

The partition can ship as data rather than as a rule. A committed JSON map from module path to chunk name is read by `rsbuild.cdn.mjs`, which turns it into one enforced asynchronous cache group per chunk; externals and entry chunks stay out of the map, and modules missing from it fall

back to today's rules. A script regenerates the map from an analysis build and the greedy heuristic, in seconds, for example from a weekly job or whenever components change, and opens a pull request. A check in CI recomputes the usage signatures and flags modules whose usage changed since the map was computed, so that a stale map cannot ship unnoticed. A significant change of the map is measured with the ADR's harness before release.

9 Related work

The trade-off between bundling and unbundling under HTTP/2 was measured by Khan Academy, whose individual files compressed worse and loaded slower than packages [6]. Next.js and Gatsby replaced a single commons chunk with granular chunks bounded by size and request count [7], a rule-based version of the trade-off this paper optimises. Rspack's SplitChunks plugin, like webpack's, exposes the knobs used here: cache groups, `minChunks`, and `enforce` [8]. ADR 0001 searched the one-parameter family of `minChunks` thresholds [1]. On the optimisation side, the problem resembles clustering and set partitioning; LPT scheduling [12] is the classic heuristic for balance, which this objective does not reward, and CP-SAT [10, 11] solves the integer program at this size in minutes.

10 Conclusion

Computing which modules share a chunk, instead of choosing a threshold, makes Atomic's pages ready up to 605 ms sooner on a mid-range phone than the ADR's `minChunks: 10`, and up to 1.06 s sooner than the build before the ADR. The gain comes from requests, the algorithm hardly matters, and a cost model fitted on the ADR's measurements of October 15 predicted it within 140 ms. The measurement mattered most where the model was blind: externals must keep their own chunks, and the partition must follow the code as it changes. Feeding the model real page compositions is the next step.

Acknowledgements

The prototype, the experiments and this paper were produced with Claude Code, Anthropic's coding agent, working in the ui-kit repository with the author.

A Reproduction

Everything is on the branch `prototype/atomic-cdn-chunk-partition` of the ui-kit repository, in `packages/atomic/scripts/prototype-chunk-partition`:

- `pages.mjs`, `analyze.mjs`, `fit.py`, `solve.py`, `rsbuild.proto.mjs`, `measure.mjs`, `check-registration.mjs`: the pipeline of Section 5.
- `solve-variants.py`, `greedy.py`, `gen-pages.py`, `eval-map.py` and `STRATEGIES.md`: Sections 6.5 and 6.6.
- `MEASUREMENT.md` and `report-measured.mjs`: Section 6.7 and its figures.

The environment variables of `rsbuild.proto.mjs` select the build: `PROTO_CHUNK_MAP` enforces a map, `PROTO_NO_CONCAT` turns module concatenation off for the analysis build, and `PROTO_NO_SHARED` rebuilds the pre-ADR state; `PROTO_OUT` and `PROTO_STATS` set the output folder and the stats file. A typical sequence, run from `packages/atomic`:

```
PROTO_NO_CONCAT=1 PROTO_OUT=out/analysis PROTO_STATS=out/analysis/stats.json \
  node scripts/prototype-chunk-partition/rsbuild.proto.mjs
node scripts/prototype-chunk-partition/pages.mjs out/pages.json
node scripts/prototype-chunk-partition/analyze.mjs out/analysis out/pages.json \
  out/model.json out/baseline
python3 scripts/prototype-chunk-partition/solve.py out/model.json cost.json out/solver
PROTO_CHUNK_MAP=out/solver-chunk-map.json PROTO_OUT=out/solver \
  PROTO_STATS=out/solver/stats.json node scripts/prototype-chunk-partition/rsbuild.proto.mjs
node scripts/prototype-chunk-partition/measure.mjs out/solver out/pages.json cost.json
```

The harness runs (`run.mjs`, `server.mjs`, `measure.mjs`, `waterfall-json.mjs`, `analyze.py`) and every results file are kept with the ADR's harness on the author's machine, in `atomic-perf-harness/partition-check`.

B Verbatim logs

Solver runs of October 31:

```
opt-all:        status FEASIBLE in 150 s; objective 7117736 B-eq, bound 6564517 (gap 7.8%)
opt-holdout:    status FEASIBLE in 150 s; objective 1927310 B-eq, bound 1868131 (gap 3.1%)
opt-lam-half:   status FEASIBLE in 150 s; objective 6897171 B-eq, bound 6415420 (gap 7.0%)
opt-lam-double: status FEASIBLE in 150 s; objective 7433141 B-eq, bound 6738640 (gap 9.3%)
```

Solver runs of November 3:

```
long, warm start:  status FEASIBLE in 900 s; objective 7093429 B-eq, bound 6594300 (gap 7.0%)
fresh map:         status FEASIBLE in 150 s; objective 7162714 B-eq, bound 6343992 (gap 11.4%)
```

Measurements of the builds of October 31, 42 pages:

```
pre-adr:  532 js files, 644.0 KB gz total; 167.6 files, 207.7 KB gz, 1.807 s modelled
baseline: 433 js files, 613.9 KB gz total;  83.1 files, 187.2 KB gz, 1.319 s modelled
opt-all:  215 js files, 521.9 KB gz total;  14.0 files, 160.5 KB gz, 0.870 s modelled
```

Harness medians, mobile, ms, for the search box, typical, rich and commerce pages:

```
run 1  before 2827 4181 5244 6087 | N=10 2673 3629 4679 5548 | solver 2716 3903 4686 5024
run 2                             N=10 2690 3684 4719 5665 | solver 2685 3367 4230 5165
run 3  (not used)                 N=10 2704 3696 4753 5593 | solver 2661 3379 4198 5134
run 4                             N=10 2695 3702 4702 5608 | solver 2667 3395 4192 5106
run 5  before 2862 4174 5223 6144 | N=10 2715 3688 4765 5601 | solver 2676 3388 4160 5148
```

C Timeline and spend

- **October 31, 2026.** Survey of optimisation opportunities in the repository; prototype pipeline, two model fixes, four solves (all pages, held-out, half and double λ), builds and result pages. About five hours, four solver runs of 150 s and seven builds of 2 s.
- **November 3, 2026.** Algorithm comparison and synthetic training sets (solver runs of 900 s, three of 300 s and one of 150 s, 33 minutes in all); five harness runs, 528 page loads in 32 minutes; two fixes (externals kept apart, local externals in the registration check); write-ups and this paper. About three hours.

Fixes between harness runs are listed in Section 6.7. No result was discarded except run 3, whose builds came from two commits.

References

[1] Coveo UI-Kit team. *ADR 0001: Group shared modules in one CDN chunk*. ui-kit repository, `packages/atomic/docs/adr/0001-group-shared-modules-in-one-cdn-chunk.md`, October 2026.

[2] Google. *Lighthouse network throttling and CPU throttling*. https://github.com/GoogleChrome/lighthouse/blob/main/docs/throttling.md, accessed November 3, 2026.

[3] Chrome for Developers. *Avoid chaining critical requests*. https://developer.chrome.com/docs/lighthouse/performance/critical-request-chains, accessed November 3, 2026.

[4] HTTP Archive. *Web Almanac 2024: JavaScript*. https://almanac.httparchive.org/en/2024/javascript.

[5] A. Osmani. *JavaScript start-up optimization*. web.dev, https://web.dev/articles/optimizing-content-efficiency-javascript-startup-optimization.

[6] C. Silverstein. *Forgo JS packaging? Not so fast*. Khan Academy Engineering, December 2015. https://calendar.perfplanet.com/2015/forgo-js-packaging-not-so-fast/.

[7] H. Djirdeh. *Improved Next.js and Gatsby page load performance with granular chunking*. web.dev, https://web.dev/articles/granular-chunking-nextjs.

[8] Rspack. *SplitChunksPlugin*. https://rspack.rs/plugins/split-chunks-plugin.

[9] M. Thomson and C. Benfield, editors. *HTTP/2*. RFC 9113, June 2022.

[10] L. Perron and F. Didier. *CP-SAT*. Google OR-Tools, version 9.15. https://developers.google.com/optimization/cp/cp_solver.

[11] L. Perron, F. Didier and S. Gay. The CP-SAT-LP solver. In *29th International Conference on Principles and Practice of Constraint Programming (CP 2023)*, LIPIcs 280, pages 3:1–3:2, 2023.

[12] R. L. Graham. Bounds on multiprocessing timing anomalies. *SIAM Journal on Applied Mathematics*, 17(2):416–429, 1969.